

Web Brute-Force & CSRF Attack Simulation

Cyber Security Project
UNIME Department of Engineering
Professor Francesco Longo
Student Rana Sanjideh

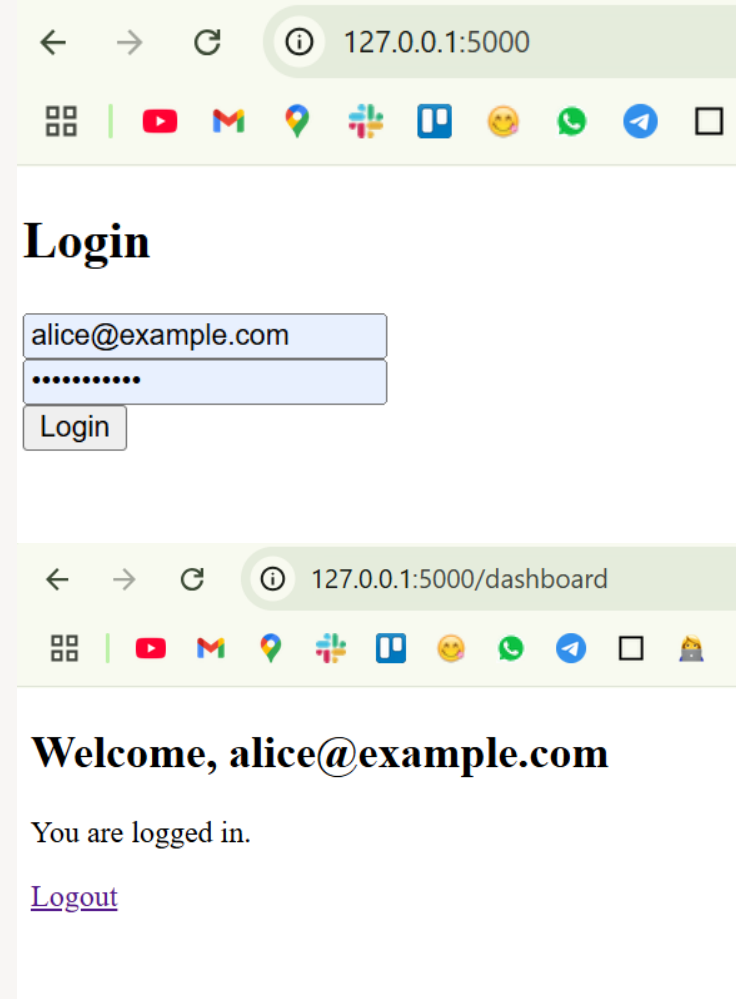


Concepts

- Authentication & Passwords: Used hashed credentials (SHA-256) for secure login.
- Guessing Attacks: Simulated brute-force login using automated scripts.
- Rate Limiting: Blocked repeated login attempts with Flask-Limiter.
- Bot Detection (CAPTCHA): Used math CAPTCHA to prevent automated login bots.
- Cross-Site Request Forgery (CSRF): Simulated CSRF attack by auto-submitting hidden forms.
- CSRF Defenses (Tokens): Protected email-change forms with CSRF tokens (Flask-WTF).
- SameSite Cookie Policy: Enforced SameSite=Lax to block cross-origin form abuse.
- Session Handling: Maintained login status using Flask's session.
- Fail Securely: All sensitive routes fail unless token/session is valid.
- Minimize Attack Surface: Sensitive routes only accessible after secure login.

Login System and Dashboard

- Created a Flask web app with a /login route.
- Users submit email and password.
- Passwords compared using SHA-256 hash as hashed passwords protects against database leaks. This setup is vulnerable to brute-force without defense
- After login, user is redirected to a dashboard.
- Flask's session tracks logged-in user.



Brute-Force Attack Simulation

test common passwords against the login route which shows without defense, attacker can find valid credentials.

```
FAILED: alice@example.com:123456
SUCCESS: alice@example.com:password123
FAILED: bob@example.com:123456
SUCCESS: bob@example.com:qwerty
```

```
from flask import Flask, request, render_template, redirect, url_for, session
import hashlib

app = Flask(__name__)
app.secret_key = 'your_secret_key'

users = {
    "alice@example.com": hashlib.sha256("password123".encode()).hexdigest(),
    "bob@example.com": hashlib.sha256("qwerty".encode()).hexdigest()
}

@app.route('/', methods=['GET', 'POST'])
def login():
    message = ""
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']
        hashed_pw = hashlib.sha256(password.encode()).hexdigest()

        if email in users and users[email] == hashed_pw:
            session['user'] = email
            return redirect(url_for('dashboard'))
        else:
            message = 'Invalid email or password.'

    return render_template('login.html', message=message)

@app.route('/dashboard')
def dashboard():
    if 'user' not in session:
        return redirect(url_for('login'))
    return render_template('dashboard.html', email=session['user'])

@app.route('/logout')
def logout():
    session.clear()
    return redirect(url_for('login'))

if __name__ == '__main__':
    app.run(debug=True)
```

Add CAPTCHA & Rate Limiting

- Implemented a simple math CAPTCHA using random numbers.
- Limited login attempts to 5 per minute per IP using `flask_limiter`
- CAPTCHA defends against bots
- Rate Limiting throttles brute-force attempts

Together, they make guessing attacks harder, even if password is weak.

Add CAPTCHA & Rate Limiting

```
# Rate limiting: 5 Login attempts per minute per IP
limiter = limiter(
    get_remote_address,
    app=app,
    default_limits=["5 per minute"]
)

users = {
    "alice@example.com": hashlib.sha256("password123".encode()).hexdigest(),
    "bob@example.com": hashlib.sha256("qwerty".encode()).hexdigest()
}

@app.route('/', methods=['GET', 'POST'])
@limiter.limit("5 per minute") # Limit brute-force
def login():
    message = ''

    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']
        user_captcha = request.form['captcha_answer'].strip()

        # Check CAPTCHA first
        if user_captcha != session.get('captcha_answer'):
            message = 'Invalid CAPTCHA.'
        else:
            hashed_pw = hashlib.sha256(password.encode()).hexdigest()
            if email in users and users[email] == hashed_pw:
                session['user'] = email
                return redirect(url_for('dashboard'))
            else:
                message = 'Invalid email or password.'

        # Generate CAPTCHA on every GET and failed POST
        num1 = random.randint(1, 10)
        num2 = random.randint(1, 10)
        captcha_answer = str(num1 + num2)
        captcha_question = f"What is {num1} + {num2}?"
        session['captcha_answer'] = captcha_answer

    return render_template('login.html', message=message, captcha_question=captcha_question)

@app.route('/dashboard')
def dashboard():
    if 'user' not in session:
        return redirect(url_for('login'))
    return render_template('dashboard.html', email=session['user'])
```

← → ↻ ⓘ 127.0.0.1:5000

🗲️ | 📺 📧 📍 🌐 📺 😊 📞 📩

Login

What is 6 + 8?

← → ↻ ⓘ 127.0.0.1:5000

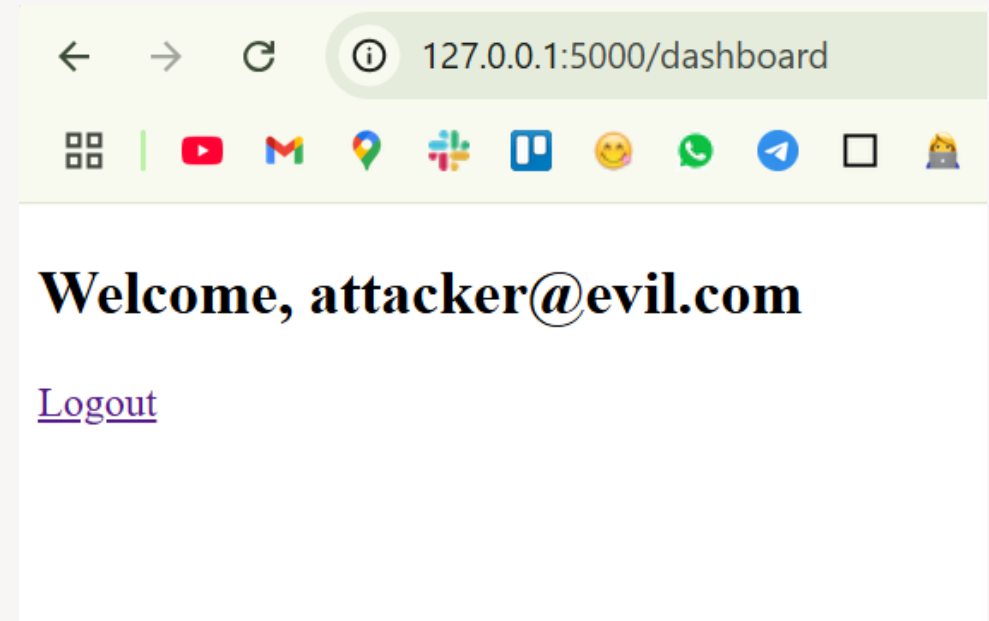
🗲️ | 📺 📧 📍 🌐 📺 😊 📞 📩

Too Many Requests

5 per 1 minute

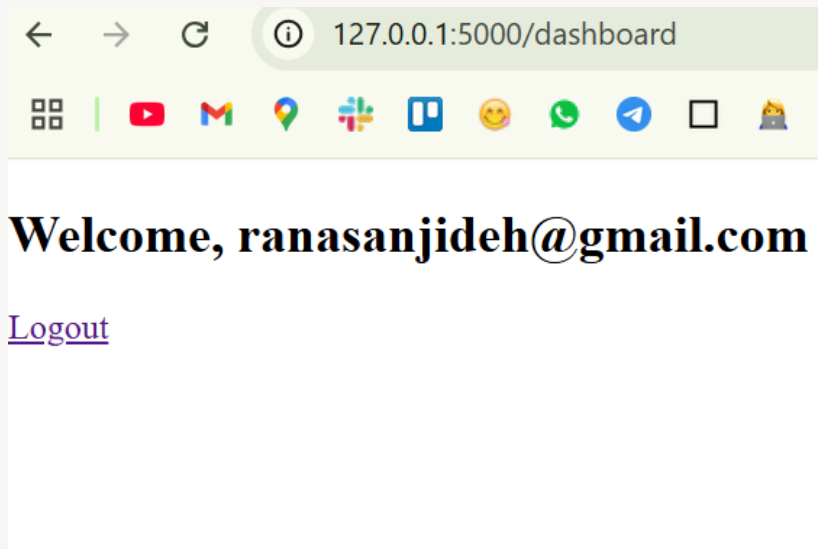
CSRF Attack Simulation

- Built attacker.html to auto-submit a hidden form that changes the user's email.
- If user is logged in, their session cookie gets used unknowingly.
- It is a Classic Cross-Site Request Forgery. Shows how attackers abuse authenticated sessions in web apps.



Defend with CSRF Tokens & SameSite Cookies

- Added Flask-WTF CSRF protection on forms.
- Set SESSION_COOKIE_SAMESITE = 'Strict' in Flask config.
- CSRF tokens for state-changing operations.
- SameSite cookies prevent CSRF if the browser respects cookie policy.
- Ensures email change can only be triggered from same origin and valid token.



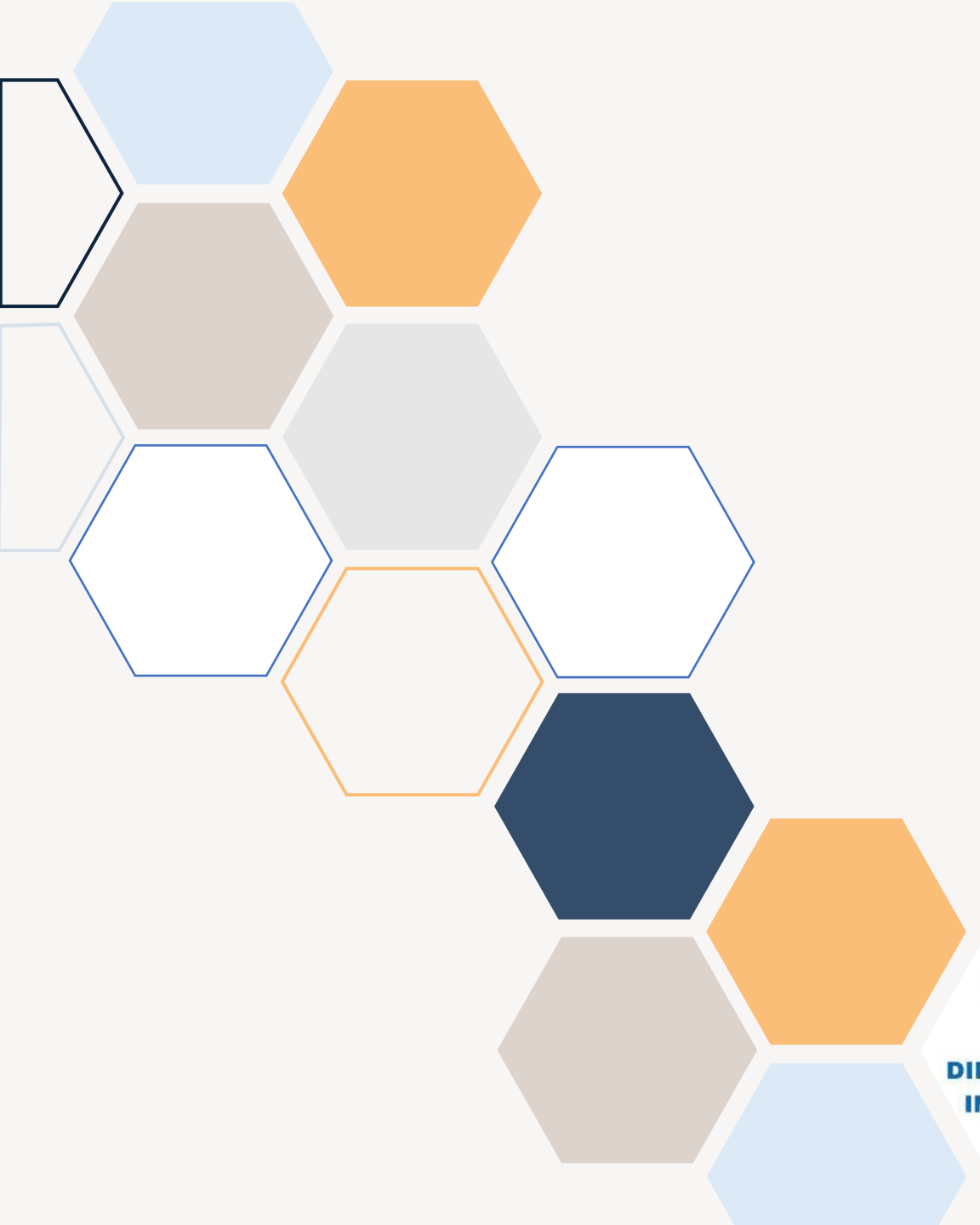
Change Email

Email changed to attacker@evil.com

[Back to Dashboard](#)

Results

- Login Page: login with valid email/password-CAPTCHA works and blocks incorrect answers-Rate limiting
- Dashboard: After login, email is shown on /dashboard
- Brute-force Attack: brute_force.py shows SUCCESS or FAILED, which means we find password or not
- CSRF Simulation: runs silently before defense and victim's email gets changed but after defense, we get a CSRF Error
- Change Email Page: CSRF protection token which means If it's missing or wrong → reject the request (**CSRF error**) - Submits new email and updates session

A decorative pattern of hexagons in various colors (blue, orange, grey, white) arranged in a honeycomb-like structure, primarily located on the left side of the slide.

Thank you

Rana Sanjideh

RanaSanjideh@gmail.com



**DIPARTIMENTO
INGEGNERIA**
UniMe